Teaching the sorting factory workshop

22 October 2026 · Four hours · First semester · No programming or robotics prerequisites

The class learns by predicting a small change, running it, and explaining visible results. Use the student notebook as the main route. The separate instructor notebook adds answers beside the same activities. Pair one student who types with one who predicts and have them swap roles each activity. Success means an explained, tested result, not finishing every optional challenge.

Prepare before the class

Give every pair an extracted student ZIP and an already prepared Python environment. Jupyter and Matplotlib need dependencies; the lesson works without internet after preparation. The plain Python scripts and HTML animation need only Python 3.11 or newer. Do not spend classroom time installing packages.

On the classroom computer prepare a dedicated environment, separate from any robot or ROS environment:

```
python3 -m venv .venv-workshop
```

```
.venv-workshop/bin/python -m pip install -r notebooks/requirements.txt
```

```
.venv-workshop/bin/python tools/verify_notebooks.py
```

```
.venv-workshop/bin/python exercises/make_student_bundle.py --output docs/student-workshop.zip
```

Copy the ZIP to student machines and extract it. Prepare the same dependencies on those machines before class; a virtual environment copied from another machine is not portable. On Windows the environment Python is `.venv-workshop\Scripts\python.exe`. Install using that Python and the same requirements file.

Launch from the extracted folder:

```
.venv-workshop/bin/python notebooks/launch.py
```

If Jupyter is already prepared under another interpreter, run that interpreter followed by `notebooks/launch.py`. The launcher opens the student notebook in JupyterLab with the bundle folder as its root. On Windows use the prepared Python executable with the same argument. Make a team copy with File → Save Notebook As; retain the original. Restarting the kernel clears memory, not saved file edits. Keep the instructor notebook away from the learner ZIP.

Before students arrive, open the notebook, run setup and the first mission, and check that the bins appear. Confirm that a fresh copy runs top to bottom. Print one worksheet per pair. Project the eight introduction slides. If hardware is unavailable, use the station photograph, paper objects and notebook simulation for all rotations. Only the instructor controls physical equipment; autonomous physical sorting is not established by these notebook checks.

Introduction 25 minutes

Use the eight slide deck and its speaker notes. Ask students how they would sort one object by hand. Build observe → decide → act → check from their answers. Point out the camera, conveyor and robot in the real station photograph. Define sensor as something that measures and actuator as something that changes the world. Ask what should happen when a camera report is missing. Finish by showing exactly how to open a team copy, click a code cell, press Shift+Enter, find output and save.

A First instructions 25 minutes

Allow 5 minutes for the one-line worked example and notebook controls, 8 minutes for reading and running the original mission, 8 minutes for individual changes and 4 minutes for pair explanation. Ask “What will the last object number be?” before running. Point to a single = storing a value, a double == asking a question, quotes marking text and four spaces grouping a loop body. Avoid introducing concepts that have not appeared yet.

Expected original output: object numbers 1, 2, 3 and decision left. Five repetitions produce 1–5; blue chooses right; yellow chooses reject. Green extension uses another `elif COLOR == "green":`

before else and an indented `destination = "left"`. First hint: change only the value on the right of =. Second hint: point to the quoted color. Ask a student to explain why changing a name label does not change the repeat count.

If a quote is missing, read the last error line aloud, repair one quote and rerun. If indentation changes, compare the line with the clean original. Demonstrate this once; do not treat an error as failure.

B Camera information 35 minutes

Spend 8 minutes comparing the real photograph crops and their human labels, 12 minutes on the five simulated reports, 10 minutes on actual versus reported color, and 5 minutes sharing explanations. The photograph contains real colored objects; the lists are controlled simulated sensor reports, not an implemented image classifier.

Default counts: left 2, right 2, reject 1. Changing the second blue report to red produces 3, 1, 1. Adding green without a rule adds a reject. Reverse the two destination variables to flip individual actions. For green insert an elif before else; preserve fallback rejection. In the actual/report experiment, changing only the first report from red to blue sends an actual red object right. Changing it to None rejects it. A believable incorrect report is not detectable by this routing rule alone.

First hint: trace one list item through its condition. Second hint: underline the report and destination separately. Ask “Did the program do the wrong thing, or did it receive wrong information?” Invite lighting, shadows and occlusion examples. Do not claim the human crop labels are measured recognition accuracy.

Break 10 minutes

Save work. Display the three rotation groups and their starting stations. Keep a clean original ready for students who need recovery.

C Three rotations 45 minutes

Run three slots of exactly 15 minutes. Group 1: robot → conveyor → camera. Group 2: conveyor → camera → robot. Group 3: camera → robot → conveyor. In each slot allow 2 minutes to describe the task, 9 minutes to try it, 3 minutes to explain an observation and 1 minute to rotate.

At robot use notebook 02 to plan left/left/right, then add reject and an invalid command. Default left/right/left gives left 2/right 1; changed left/left/right has the same totals and a different order. Middle becomes reject. Ask what a real arm needs beyond a label: coordinates relative to a reference, reachable positions, a safe path and gripper timing. A recorded action is not proof of a completed placement.

At conveyor ask one partner to call move → stop → observe → decide → pick while the other moves paper objects. Ask what changes if pick happens before stop. At camera compare crops, hide part of one object and discuss uncertain reports. These are complete fallback activities; live demonstrations supplement them only when the instructor has qualified the station. Keep students outside any moving workspace.

D Integrated factory 45 minutes

Use 8 minutes to trace the worked two-object example, 12 minutes for five-object predictions and baseline run, 15 minutes for rule changes, and 10 minutes to compare order and counts. Explain a mapping only now, using red → left and blue → right. Point out braces, colons and commas in the code.

Baseline: left 2/right 2/reject 1, five advances and actions left/right/left/reject/right. A reversed rule has identical totals on this balanced input but different actions. This is a useful discovery: totals alone can hide a mistake. Ask teams to invent an asymmetric input. Every cycle stops the belt before reading or picking, and stops again on completion. The helper recreates state each time the cell runs.

Green extension adds `"green": "left"` and at least one green report. None and unknown remain rejected with the normal red/blue/green rules. An invalid destination such as middle is rejected. The count assertion checks that objects are not lost; it does not prove each destination is correct.

E Independent experiments 40 minutes

Give 5 minutes to choose two challenges and write predictions, 25 minutes to run and record them, and 10 minutes to swap tests with another pair. Offer one hint at a time. Early finishers design a test that catches swapped rules; they do not need physical hardware.

Suggested answers: adding None, unknown, 42 and empty text to baseline produces left 2/right 2/reject 5. A middle destination for red makes baseline left 0/right 2/reject 3. A ten-report run must total ten. Green without a rule rejects; with a valid rule it reaches that bin. Swapping an unbalanced input changes totals. An assertion like `assert factory.counts["reject"] == 1` is useful on baseline but insufficient alone.

Closing 15 minutes

Give pairs 5 minutes to prepare a result, 7 minutes for several short presentations and 3 minutes for a final explanation and save. Each presentation includes a prediction, a changed rule, observed evidence and one real-world limitation. Invite quieter students to explain a single route. Collect worksheets as formative evidence, not a speed contest.

If the notebook environment fails

Use the prepared plain Python route. Open a terminal, then run one command at a time. The scripts locate their own imports, so they also work when invoked with an absolute path from another directory.

```
python3 exercises/check_setup.py
```

```
python3 exercises/01_python.py
```

```
python3 exercises/03_vision.py
```

```
python3 exercises/02_robot.py
```

```
python3 exercises/04_sort.py
```

```
python3 exercises/05_visual_sort.py
```

Students edit the marked STUDENT AREA, save, and rerun the same command. Open sorting-result.html in a browser for the fallback animation; it needs no network. Use student-exercises.md for explanations and print the station photo if necessary. If no computers work, enact the same decision rules with paper reports and three labeled bins and record predictions on the worksheet.